

Module 1: Working with Macros in Excel

Learning Objective

By the end of this module, learners should be able to:

- Understand what macros are and their purpose in Excel
- Record, run, and edit a macro
- Apply macros to automate repetitive data cleaning and formatting tasks
- Save and use a macro-enabled workbook

1. Introduction to Macros

A **macro** is a sequence of actions recorded in Excel to automate repetitive tasks — such as cleaning data, formatting sheets, or creating reports.

Macros are powered by **VBA (Visual Basic for Applications)**, but users can record them without writing any code.

Think of a macro as a “*recorded shortcut*” for multi-step tasks.

2. Enabling the Developer Tab

Before working with macros, you must ensure that the **Developer tab** is visible on the Excel ribbon.

Steps:

1. Go to **File** → **Options** → **Customize Ribbon**
2. On the right-hand list, check the box for **Developer**
3. Click **OK**

You should now see the **Developer** tab added to the top ribbon.



3. Recording a Macro

Example Scenario:

You have a dataset in the “**Macro_Demo**” sheet (in `Excel_Automation_Class.xlsx`) that contains:

- Inconsistent name capitalization
- Extra spaces
- Duplicate entries

You will record a macro that cleans and formats this data automatically.



Step-by-Step Instructions

1. **Open the Macro_Demo sheet** in your workbook.
2. Click **Developer** → **Record Macro**.
3. In the dialog box:
 - **Macro name:** `CleanData`
 - **Shortcut key:** (Optional, e.g., `Ctrl + Shift + C`)
 - **Store macro in:** `This Workbook`
 - **Description:** Automates data cleaning
4. Click **OK** — Excel is now *recording* all your actions.



Perform the Cleaning Steps

While recording:

1. **Select the dataset** (e.g., `A1:C6`).
2. Go to **Data** → **Remove Duplicates**, and click **OK**.
3. Use formulas to **trim extra spaces**:
 - In a new column, type `=TRIM(A2)` and drag down.
 - Copy the cleaned results and paste as **values** over the original column.
4. Convert all names to **Proper Case**:
 - Use `=PROPER(A2)` and fill down.
 - Paste values back.
5. Select your data and format it as a **Table** (Home → Format as Table).
6. Apply **bold headers** and add borders (Home → Font group).

Stop Recording

- Go to **Developer** → **Stop Recording**

Your macro has now been saved inside the workbook.

4. Running a Macro

There are several ways to run a macro:

Option 1: From the Developer Tab

1. Go to **Developer** → **Macros**
2. Select `CleanData`
3. Click **Run**

Option 2: Using a Keyboard Shortcut

If you assigned one during recording, simply press it (e.g., `Ctrl + Shift + C`).

Option 3: Assign to a Button

1. Go to **Insert** → **Shapes** → **Button**
2. Draw a button on your sheet
3. Right-click → **Assign Macro** → select `CleanData`
4. Click the button to run the macro anytime.

5. Viewing or Editing the Macro Code (VBA Editor)

To see the VBA code Excel generated:

1. Go to **Developer** → **Macros** → **Select** `CleanData` → **Edit**
2. The **VBA Editor** will open.
3. You'll see code similar to this:

```
Sub CleanData()  
    ' This macro automates cleaning of the employee dataset  
    Cells.Select  
    Selection.RemoveDuplicates Columns:=Array(1, 2, 3), Header:=xlYes  
    Columns("A").Select  
    Selection.Replace What:=" ", Replacement:=" ", LookAt:=xlPart  
    Selection.Font.Bold = True  
    Range("A1:C1").Borders.LineStyle = xlContinuous  
End Sub
```

✖ You can modify or enhance the macro later — for example, to include formatting or conditional logic.

6. Saving a Workbook with Macros

To ensure your recorded macros are saved:

1. Go to **File** → **Save As**
2. Under “Save as type,” choose
Excel Macro-Enabled Workbook (*.xlsm)
3. Name your file, e.g., `Excel_Automation_Project.xlsm`
4. Click **Save**

7. Practical Exercise

Objective: Practice automating a data cleaning task.

Instructions:

1. Open **Excel_Automation_Class.xlsx** and go to **Macro_Demo** sheet.
2. Record a macro named “**CleanData**” that:
 - Removes duplicate entries
 - Trims spaces
 - Converts names to proper case
 - Formats the range as a table
3. Stop recording.
4. Assign your macro to a button and run it.
5. Save the file as a **Macro-Enabled Workbook (.xlsm)**.

8. Trainer's Notes / Teaching Tips

- Emphasize that macros replicate *every exact step*, including errors — learners should perform actions carefully while recording.
- Demonstrate a short, simple macro first (e.g., formatting headers).
- Then, demonstrate the complete cleaning process.
- Remind learners to **always save a backup** before running macros on important data.
- If macros are disabled, go to **File** → **Options** → **Trust Center** → **Macro Settings** to enable them for the session.

9. Summary

Step	Task	Description
1	Enable Developer Tab	Allows access to macro tools
2	Record Macro	Captures your Excel actions
3	Perform Cleaning Steps	Example: remove duplicates, trim text, format table
4	Stop Recording	Ends macro recording
5	Run Macro	Replay your automation
6	Edit Macro	Adjust VBA code if needed
7	Save Workbook	Use <code>.xlsm</code> to keep your macros

Key Takeaways

- Macros automate tasks — record once, reuse anytime.
- VBA is Excel's programming engine for macros.
- Always test your macros on sample data before applying to live datasets.
- Save macro-enabled files properly to retain functionality.